



EMERITUS

Learn. From the world's best.

Python Fundamentals – Part II



Numpy

NumPy: Fast and Efficient Arrays

A Powerful Library for Numerical Computing

NumPy helps with numerical computing using arrays and matrices

Why Use NumPy?

- **Fast:** Written in C, much quicker than Python lists
- **Concise:** Simple, readable syntax for array operations
- **Essential:** Backbone of libraries like SciPy, Pandas and scikit-learn

NumPy Arrays and Structures

Optimised Storage and Computation

NumPy Arrays

- Uses *ndarray*, a fast and efficient multi-dimensional array
- Ideal for storing and processing numerical data

Key Features

- Stores homogeneous data types
- Uses memory efficiently
- Supports various types (integers, floats, etc.)

NumPy Indexing and Slicing



Accessing Elements

Retrieve individual elements using indexing, just like Python lists



Selecting Subarrays

Use slicing to extract specific sections from a larger array for better data handling

NumPy Indexing and Slicing

Example

```
import numpy as np

# Create a 1D array
arr1d = np.array([1, 2, 3, 4, 5])

# Access the first element (index 0)
print(arr1d[0]) # Output: 1

# Access the last element (index -1)
print(arr1d[-1]) # Output: 5

# Access a range of elements (from index 1 to 3, exclusive)
print(arr1d[1:4]) # Output: [2 3 4]

# Create a 2D array
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Access the element at row 1, column 0
print(arr2d[1, 0]) # Output: 4
```

NumPy Arithmetic Operations

Efficient Mathematical Operations on Arrays

Element-wise Operations

Perform addition, subtraction, multiplication and division on arrays, applying them to each element

Broadcasting

Expand smaller arrays to match larger ones, enabling operations on different shapes

NumPy Arithmetic Operations

Perform Mathematical Operations on Arrays with Ease

Element-wise Operations

```
import numpy as np  
  
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
  
print("Addition:", arr1 + arr2)  
print("Subtraction:", arr1 - arr2)  
print("Multiplication:", arr1 * arr2)  
print("Division:", arr1 / arr2)
```

NumPy Arithmetic Operations

Perform Mathematical Operations On Arrays with Ease

Broadcasting

```
import numpy as np
```

```
# Scalar broadcasting a = np.array([1, 2, 3])
```

```
b = 2 # Scalar
```

```
result1 = a + b # Broadcasting the scalar 'b' to match the shape of 'a'
```

Universal Functions in NumPy

Built-in Mathematical Functions for Arrays

Trigonometric Functions

NumPy provides functions like sine, cosine and tangent to perform trigonometric calculations on array elements

|X|

Exponential and Logarithmic Functions

Use NumPy's universal functions to apply exponentiation and logarithmic operations efficiently to arrays

x^1

Universal Functions in NumPy

|X| Trigonometric Functions

```
import numpy as np

# Define angles in radians
angles_rad = np.array([0, np.pi/2, np.pi])

# Calculate sine, cosine, and tangent
sine_values = np.sin(angles_rad)
cosine_values = np.cos(angles_rad)
tangent_values = np.tan(angles_rad)

# Print the results
print("Angles (radians):", angles_rad)
print("Sine values:", sine_values)
print("Cosine values:", cosine_values)
print("Tangent values:", tangent_values)
```

Universal Functions in NumPy

X¹ Exponential and Logarithmic Functions

```
import numpy as np

# Define an array of values
values = np.array([1, 2, 3, 4, 5])

# Calculate exponential of each value
exponential_values = np.exp(values)

# Calculate natural logarithm of each value
log_values = np.log(values)

# Calculate base-10 logarithm of each value
log10_values = np.log10(values)

print("Original Values:", values)
print("Exponential Values:", exponential_values)
print("Natural Logarithm:", log_values)
print("Base-10 Logarithm:", log10_values)
```

NumPy Array Manipulation Functions

Modify and Organise Arrays Efficiently



NumPy Array Manipulation

Reshaping Arrays: Reshape Method

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6]) # 1D array  
  
# Reshape to a 2D array (2 rows, 3 columns)  
  
reshaped_arr = arr.reshape(2, 3)  
  
print(reshaped_arr)
```

Linear Algebra with NumPy

Key Matrix Operations



Matrix Multiplication

NumPy's `dot()` function efficiently handles matrix multiplication, a core linear algebra operation

Matrix Inversion

The `linalg` module allows easy computation of matrix inverses, essential for solving linear equations

Matrix Decomposition

NumPy supports eigenvalue and singular value decomposition, breaking matrices into simpler components

Matrix Multiplication Example

```
import numpy as np

# Define two matrices
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Perform matrix multiplication using the @ operator
result1 = matrix1 @ matrix2

# Perform matrix multiplication using the dot() method
result2 = matrix1.dot(matrix2)

print("Matrix 1:\n", matrix1)
print("Matrix 2:\n", matrix2)
print("Result using @ operator: \n", result1)
print("Result using dot() method:\n", result2)
```

Matrix Inversion Example

```
import numpy as np

# Define a square matrix
matrix = np.array([[1, 2], [3, 4]])

# Calculate the inverse of the matrix
Invers_matrix = np . Linalg . Inv(matrix)

print("Original Matrix:\n", matrix)
print("Inverse Matrix:\n", inverse_matrix)

# Verify: Multiply the matrix with its inverse
identity_matrix = matrix @ inverse_matrix

print("Identity Matrix:\n", identity_matrix)
```

Matrix Decomposition Example

```
import numpy as np

# Define a square matrix
matrix = np.array([[1, 2], [2, 1]])

# Perform eigenvalue decomposition
eigenvalues, eigenvectors = np.linalg.eig(matrix)

# Reconstruct the original matrix using eigenvalue decomposition
reconstructed_matrix = eigenvectors @ np.diag(eigenvalues) @
np.linalg.inv(eigenvectors)

print("Original Matrix:\n", matrix)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
print("Reconstructed Matrix:\n", reconstructed_matrix)
```

NumPy in Action

Key Applications



Data Analysis : Provides efficient tools for array manipulation, statistics and data exploration

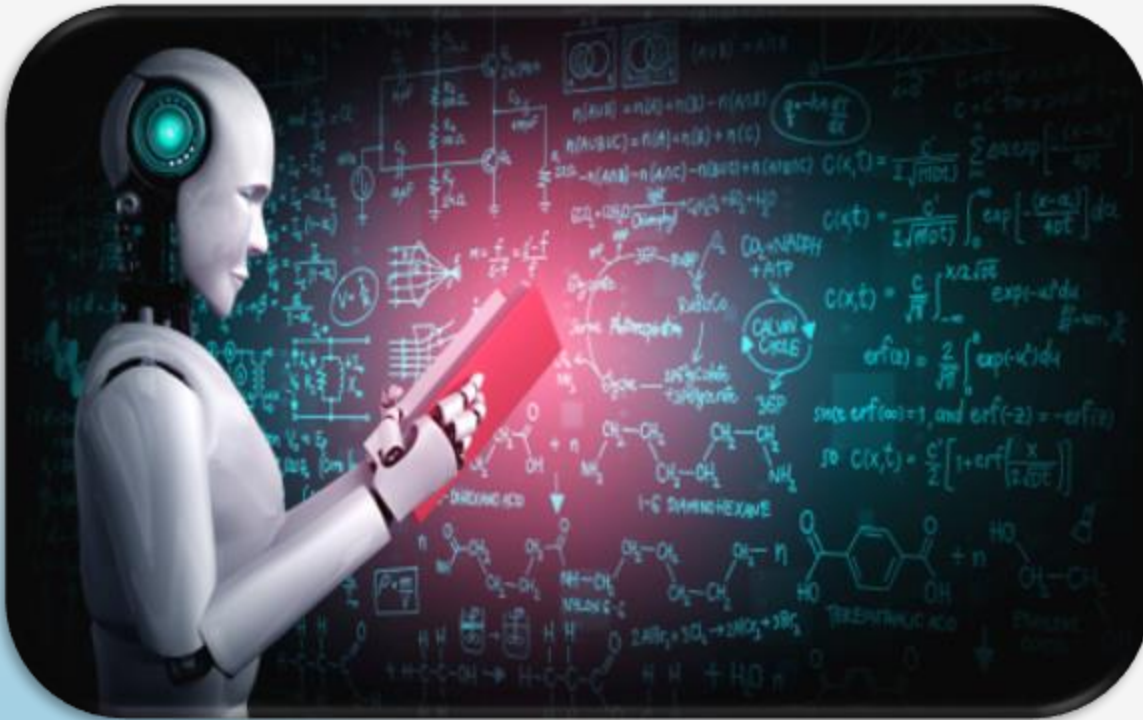


Machine Learning – Powers ML libraries for handling and processing large datasets

Pandas

Understanding Pandas

Key Features



Powerful Data Library

A Python tool for efficient data manipulation and analysis

Essential for Data Science

Simplifies data cleaning, transformation, analysis and visualisation

Why Use Pandas?

Powerful and Easy-to-Use Data Library

Efficient Data Handling

Optimised data structures ensure smooth processing of large datasets

Intuitive API

User-friendly interface makes data manipulation easy, even for beginners

Extensive Functionality

Supports data analysis, visualisation and transformation with a wide range of features

Pandas Data Structures

Series and Data Frame

Series

A one-dimensional labelled array, like a spreadsheet column, that holds different data types

DataFrame

A two-dimensional labelled table with rows and columns, ideal for structured data storage and analysis

Pandas Series

A One-Dimensional Data Structure

-  **Labelled Array:**
A fundamental Pandas structure for handling 1D labelled data
-  **Versatile Data Types:**
Supports integers, floats, strings and more
-  **Labelled Index**
Each element has a unique label for easy access
-  **Rich Functionality**
Offers various operations for data analysis and manipulation

Creating a Pandas Series

Different Ways to Initialise

From a List → `pd.Series([1, 2, 3, 4])`

From a Dictionary → `pd.Series({'a': 1, 'b': 2, 'c': 3})`

With a Custom Index → `pd.Series([1, 2, 3], index=['x', 'y', 'z'])`

Accessing Elements in a Pandas Series

Retrieve Data Easily

- Use labels or positions to retrieve values
- Supports slicing like Python lists for easy selection

```
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
# Access by label  
print(s['a'])
```

```
# Access by position  
print(s[1])
```

```
# Slicing  
print(s['b':'d'])
```

Vectorised Operations

- 1 Perform **element-wise computations** efficiently
- 2 Apply **arithmetic, comparison and mathematical functions**
- 3 Write **cleaner and more efficient code** than loops

```
s = pd.Series([1, 2, 3, 4])  
  
# Arithmetic operations  
print(s + 10)  
  
# Comparison  
print(s > 2)  
  
# Mathematical functions  
import numpy as np  
print(np.sqrt(s))
```

Essential Series Methods

Aggregation

sum(): Returns the total of all elements

mean(): Computes the average value

max() / min(): Finds the highest or lowest value

Value Counts & Custom Functions

value_counts(): Counts unique values

apply(): Applies a custom function to elements

```
s = pd.Series([1, 2, 2, 3, 4])
```

```
# Aggregation
```

```
print(s.sum())
```

```
print(s.mean())
```

```
# Value counts
```

```
print(s.value_counts())
```

```
# Apply custom function
```

```
print(s.apply(lambda x: x * 2))
```

Managing Missing Data

Key Steps:

1 Detect: Use `isna()` or `notna()` to find missing values

2 Fill: Use `fillna()` to replace missing values

3 Drop: Use `dropna()` to remove missing values

```
s = pd.Series([1, 2, None, 4])
```

```
# Detect missing values  
print(s.isna())
```

```
# Fill missing values  
print(s.fillna(0))
```

```
# Drop missing values  
print(s.dropna())
```

Pandas: DataFrame

Overview

A **DataFrame** is a 2D labelled data structure with columns that can hold different data types. It works like a dictionary of Series objects

```
import pandas as pd  
  
df = pd.DataFrame({  
    'A': [1, 2, 3],  
    'B': ['a', 'b', 'c'],  
    'C': [4.0, 5.0, 6.0]  
  
})  
print(df)
```

Reading CSV Files

Example Code

```
import pandas as pd  
  
# Read a CSV file  
df = pd.read_csv('data.csv')  
  
# Display the first 5 rows  
print(df.head())
```

Key Points:

- `pd.read_csv()` loads data from a CSV file
- `.head()` displays the first few rows

Reading Excel Files in Python

Load and View Excel Data Using Pandas

```
import pandas as pd  
  
# Read an Excel file (with sheet name)  
df = pd.read_excel('data.xlsx',  
sheet_name='Sheet1')  
  
# Display the first 5 rows  
print(df.head())
```

Key Points:

- Use `pd.read_excel()` for `.xlsx` and `.xls` files
- Specify `sheet_name` to select a specific sheet
- View data with `df.head()`

Reading JSON Files

Code Example

```
import pandas as pd  
  
# Read a JSON file  
df = pd.read_json('data.json')  
  
# Display the first 5 rows  
print(df.head())
```

Key Points:

- `pd.read_json()` loads JSON data into a Data Frame
- Automatically converts JSON into a tabular format

Reading Data from SQL Databases

Code Example

```
import pandas as pd
import sqlite3

# Connect to a SQLite database
conn = sqlite3.connect('database.db')

# Query data from a table
df = pd.read_sql_query('SELECT * FROM customers',
conn)

# Display the first 5 rows
print(df.head())
```

Key Points:

- `pd.read_sql_query()` retrieves data from an SQL database
- Requires a valid database connection

Customising Data Reading

Key Options

➔ Handling missing values

`na_values`

➔ Specifying columns to read

`usecols`

➔ Reading large files in chunks

`chunksize`

```
df = pd.read_csv('data.csv', na_values=['NA',  
'N/A'],
```

```
usecols=['col1', 'col2'])
```

Error Handling in Data Reading

Common Issues:

- ✓ File not found
- ✓ Incorrect encoding
- ✓ Unexpected delimiters

Handling Errors:

- ✓ Use try-except to manage exceptions.

```
try:  
    df = pd.read_csv('data.csv')  
except FileNotFoundError:  
    print("The file does not exist.")
```

Basic DataFrame Operations

Viewing Data:

- Use `df.head()`, `df.tail()`, `df.info()`, `df.describe()` for a quick overview

Selecting Columns:

- Access with `df['column_name']` or `df.column_name`

Filtering Rows:

- Apply boolean indexing for conditional filtering

Adding/Removing Columns:

- Add: `df['new_column'] = values`
- Remove: `df.drop('column_name', axis=1)`

Pandas Exercise

Pandas Exercise

Learning Goals

- ✓ Read CSV files
- ✓ Access columns & rows
- ✓ Filter data with conditions
- ✓ Use describe() & info()
- ✓ Add new columns
- ✓ Handle missing values
- ✓ Drop columns
- ✓ Use NumPy for data manipulation



Exercise 1: Load Data



Load CSV

Import *sales_data.csv* into a Pandas DataFrame.



Use `info()`

Get a summary of rows, columns and data types



Use `describe()`

View statistical summaries of numeric columns

Exercise 2: Accessing Data

Key Operations

1

View First 5 Rows

Use `head()` to see the first five rows of the dataset

2

Select a Column

Retrieve a specific column (e.g., Product or Price) and view unique values

3

Filter Rows

Display rows where the Sales column is greater than 500

Exercise 3: Filtering Data

- **By Region**

Show rows where the Region is "East"

- **By Price & Sales**

Display rows where Price > 100 and
Sales < 1000



Exercise 4: Handling Missing Values

1. Check for Missing Values

- Use *isnull()* and *sum()* to detect missing values

2. Fill Missing Values

- Replace missing values in the Sales column with the median

3. Drop Rows with Missing Values

- Remove rows with missing values in the Product column

Exercise 5: Adding a New Column

Add Discounted_Price Column

Create a new column,
Discounted_Price, that is 10% less
than the original Price

Display Results

Show the first 5 rows to verify
the new column

Exercise 6: Removing a Column



1. Drop Column

- Remove the **Discounted_Price** column from the DataFrame

2. Confirm Deletion

- Verify the removal by displaying the column names

Exercise 7: Using NumPy Functions



Step 1: Calculate Profit

- Subtract **Cost** from **Sales** to get **Profit**
-

Step 2: Compute Log Profit

- Apply `np.log()` to **Profit** to create **Log_Profit**

Q&A Session



Open floor for questions and clarifications.